

Aulas 8 e 9

Simulação a nível de Porta Lógica

Autores

Gabriel A. F. Souza, Gustavo D. Colletta, Leonardo B. Zoccal, Odilon O. Dutra

Unifei

Histórico de Revisões

7 de novembro de 2025	1.0	Primeira versão do documento.
-----------------------	-----	-------------------------------

Tópicos

- Introdução
- Standard Delay Format (SDF)
- Fluxo de Execução do Gate-Level Simulation
- Atividades Hands-on

Introdução

Objetivos

- Identificar o processo de **simulação em nível de portas (Gate-Level Simulation)**.
- Entender o conceito e a função da **anotação SDF (Standard Delay Format)**.
- Utilizar o **Cadence Xcelium Simulator** para executar uma simulação em nível de portas de um **contador simples**.

O que é Gate-Level Simulation (GLS)?

A **simulação em nível de portas** é uma etapa de verificação que ocorre **após a síntese lógica** ou **após o roteamento físico**.

O objetivo é **garantir que o circuito sintetizado mantenha a mesma funcionalidade do RTL**, incluindo atrasos e estruturas inseridas automaticamente.

Por que realizar a GLS?

- Verificar **estruturas de baixo consumo** inseridas durante a síntese e ausentes no RTL.
- Detectar **caminhos multiciclo** (multicycle paths) quando os testes os exercitam.
- Validar **estruturas de teste (DFT)**, como **scan chains**, inseridas após a síntese.
- Simular **padrões ATPG** (Automatic Test Pattern Generation).
- Mesmo com ferramentas avançadas como STA (Static Timing Analysis) e LEC (Logic Equivalence Checking), a GLS **continua sendo essencial** para assegurar o comportamento funcional final do chip.

Níveis de Abstração na Simulação

As simulações podem ocorrer em diferentes níveis de detalhe:

Nível	Descrição	Exemplo
Transistor	Simula o circuito físico	SPICE
Gate-Level	Usa portas e células padrão	Netlist sintetizada
RTL	Descrição comportamental	Código Verilog HDL

*Maior detalhe implica **maior esforço e tempo de simulação.***

Elementos Necessários para a Gate-Level Simulation

Para realizar a simulação em nível de portas, são necessários:

- ❶ **Netlist sintetizada ou pós-roteada** – arquivo Verilog resultante da síntese (contém as standard cells).
- ❷ **Testbench** – estímulo e monitoramento do DUT (Device Under Test).
- ❸ **Arquivo SDF (Standard Delay Format)** – contém os **atrasos extraídos** do processo de síntese.

Standard Delay Format (SDF)

O que é o SDF e a Anotação de Atrasos

- É um padrão IEEE que armazena **informações de temporização** e **atrasos** do circuito.
- Pode conter:
 - **Atrasos de módulo e de interconexão**
 - **Checks de temporização** (setup, hold, skew, etc.)
- Cada **bloco hierárquico** do design pode ter seu próprio arquivo SDF.

Processo de Anotação

- O **SDF Annotator** lê o arquivo .sdf e insere os atrasos correspondentes no modelo simulado.
- O **Xcelium Simulator** é responsável tanto pela **anotação SDF** quanto pela **execução da simulação GLS**.

Fluxo de Execução do Gate-Level Simulation

Síntese

- Executar a síntese com DFT referente ao circuito do contador:

```
cd LAB4
```

```
genus -f genus_dft_script.tcl -log base.log
```

Arquivos

- O diretório **gate_level_simulation** deve conter os seguintes arquivos para a simulação:
 - counter_netlist.v → Netlist pós-síntese com DFT
 - counter_test.v → Testbench com \$sdf_annotate
 - delays.sdf → Arquivo SDF com atrasos
 - slow_vdd1v0_basicCells.v → Biblioteca de simulação

SDF Annotate

- Modificar o testbench para incluir o SDF:

```
`ifdef SDF_TEST
initial
begin
$sdf_annotate("delays.sdf", \
counter_test.counter1,, "sdf.log", "MAXIMUM");
end
`endif
```

Sintaxe do SDF Annotate

```
$sdf_annotate("sdf_file"  
  {, module_instance}  
  {, "config_file"}  
  {, "log_file"}  
  {, "mtm_spec"}  
  {, "scale_factors"}  
  {, "scale_type"});
```

Argumentos

Argumento	Descrição
"sdf_file"	Caminho do arquivo .sdf
module_instance	Instância do DUT no testbench
"log_file"	Gera log detalhado (sdf.log)
"mtm_spec"	Seleciona atraso <i>MINIMUM</i> , <i>TYPICAL</i> ou <i>MAXIMUM</i>
"scale_factors"	Multiplicadores (ex: 1.6:1.4:1.2)
"scale_type"	Define como aplicar a escala (<i>FROM_TYPICAL</i> , <i>FROM_MAXIMUM</i> , etc.)

Simulação com xrun

```
cd gate_level_simulation
```

```
xrun -timescale 1ns/10ps \  
    counter_netlist.v counter_test.v \  
    -v slow_vdd1v0_basicCells.v \  
    -access +rwc \  
    -define SDF_TEST \  
    -sdf_verbose \  
    -mess \  
    -gui
```

Opções de execução

Opção	Função
-timescale 1ns/10ps	Define unidade e precisão de tempo
-v	Inclui biblioteca .v com as standard cells
-access +rwc	Permite leitura/escrita/conectividade
-define SDF_TEST	Define macro usada no testbench
-sdf_verbose	Gera arquivo sdf.log com detalhes de anotação
-mess	Exibe mensagens detalhadas de compilação/simulação
-gui	Abre a interface SimVision GUI

SimVision

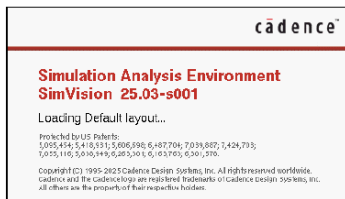
- Ao rodar o comando, o SimVision abre duas janelas:
 - Console Window
 - Design Browser
- Forçar Sinais de Teste
 - No console, execute:

```
force SE = 0
```

```
force scan_in = 0
```

```
force scan_out = 0
```

SimVision



1. SimVision™ starts with the Console and Design Browser windows opening as seen below.

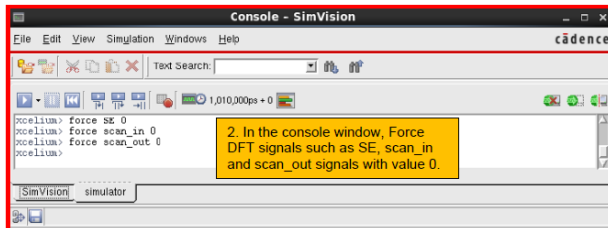


Figura 1: Inicialização da ferramenta

Simulação

- No Design Browser, selecione: counter_test
- Adicione os sinais clk, reset, count_out, etc.
- Clique em Send Selected Objects to Waveform Window.
- Executar Simulação:
 - Clique em Run na janela Console.

Simulação

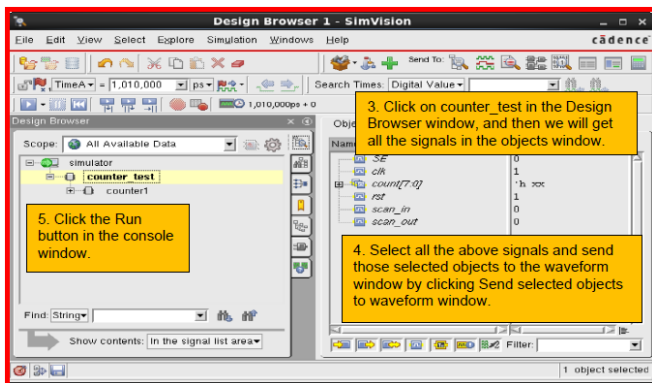


Figura 2: Execução da simulação

Simulação

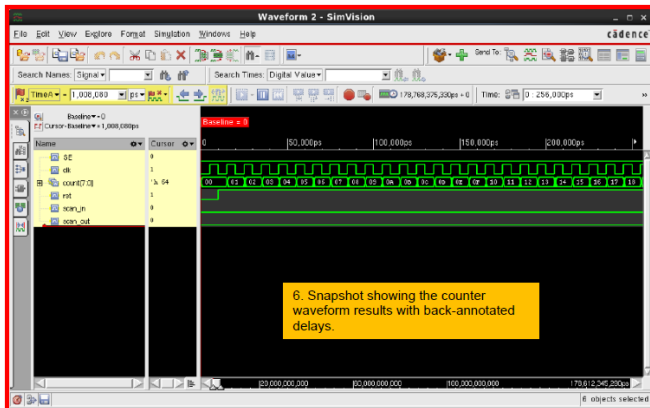


Figura 3: Resultado da simulação

Análise

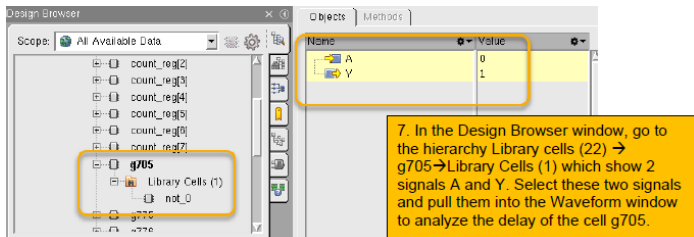


Figura 4: Análise da simulação

- Observe o contador contando corretamente:
 - Agora cada transição exibe atrasos back-annotados do arquivo SDF.
 - No exemplo, o atraso entre sinais A e Y da célula g705/INVX1 é de 0.04 ns.
 - Procurar pelo nome correspondente da célula inversora na netlist resultante da síntese.

Atividades Hands-on

Atividade

- Realizar os ajustes conforme necessário e executar o fluxo apresentado para simulação em Gate-Level.
- Documentar as etapas realizadas e analisar/comentar os resultados obtidos. Entregar documento em arquivo PDF.